Temporal Dynamics and Community Evolution in Online Social Networks

Network Science — Group Project

Team: - Virat (2022578) - Dhruv Kumar (2023202) - Ruchir Bhatowa (2022419)

Date: 2026-04-17

Abstract

(≈150 words.) Summarize: (1) the question — how do communities in an online Q&A network evolve over 6 years; (2) the dataset — Stanford SNAP sx-mathoverflow, 24,818 users, 506,550 timestamped interactions from 2009-09 to 2016-03; (3) the methods — quarterly snapshot decomposition, temporal network metrics, Louvain community detection with Jaccard/containment matching for event tracking, link prediction with four heuristic baselines vs a GraphSAGE model; (4) the headline findings — sparsification despite population growth, persistent modular structure (~0.67), dominance of preferential attachment for future-link prediction, and 862 community-evolution events classified into the six canonical Palla-style event types.

---

1. Introduction

Online communities form, mature, merge, and dissolve over time. Static snapshots of social networks hide this dynamism. The goal of this project is to characterize **how** and **at what scale** communities evolve in a real-world Q&A platform, and to contrast **structural predictions** of network science with **data-driven predictions** from graph neural networks.

Research questions. 1. How do global structural properties of the interaction graph change as the platform matures? 2. Do communities exhibit measurable evolutionary events (birth, death, merge, split, growth, shrinkage, continuation), and at what rates? 3. Can a graph neural network exploit topological structure to predict future links beyond what classical similarity heuristics provide?

---

2. Dataset and Preprocessing (Deliverable 1)

2.1 Source

We use the `sx-mathoverflow` temporal interaction graph from Stanford SNAP. Each directed, timestamped edge represents an answer-to-question (a2q), comment-on-question (c2q), or

comment-on-answer (c2a) event between two users on MathOverflow. The dataset aggregates all three interaction types.

- **Nodes (users):** 24,818
- **Temporal edges:** 506,550
- **Time span:** 2009-09-28 → 2016-03-06
- **Directed?** Yes (we build undirected graphs for community analysis; directionality is preserved in the raw DataFrame).

2.2 Temporal snapshot construction

Edges are binned into **quarterly windows** (pandas `QS` frequency). Each snapshot is the undirected interaction graph induced by the edges falling in that window, with edge weight equal to the number of interactions in the window. We discard windows with fewer than 100 edges (very early months). The final sequence is **26 snapshots** spanning 2009-Q4 → 2016-Q1.

Quarterly binning was chosen as the sweet spot between temporal resolution and per-snapshot connectivity; monthly windows produced too many disconnected graphs for meaningful community analysis, yearly windows collapsed the most interesting event structure.

Implementation: `src/data_loader.py`, `src/snapshots.py`, `notebooks/01_build_snapshots.py`.

---

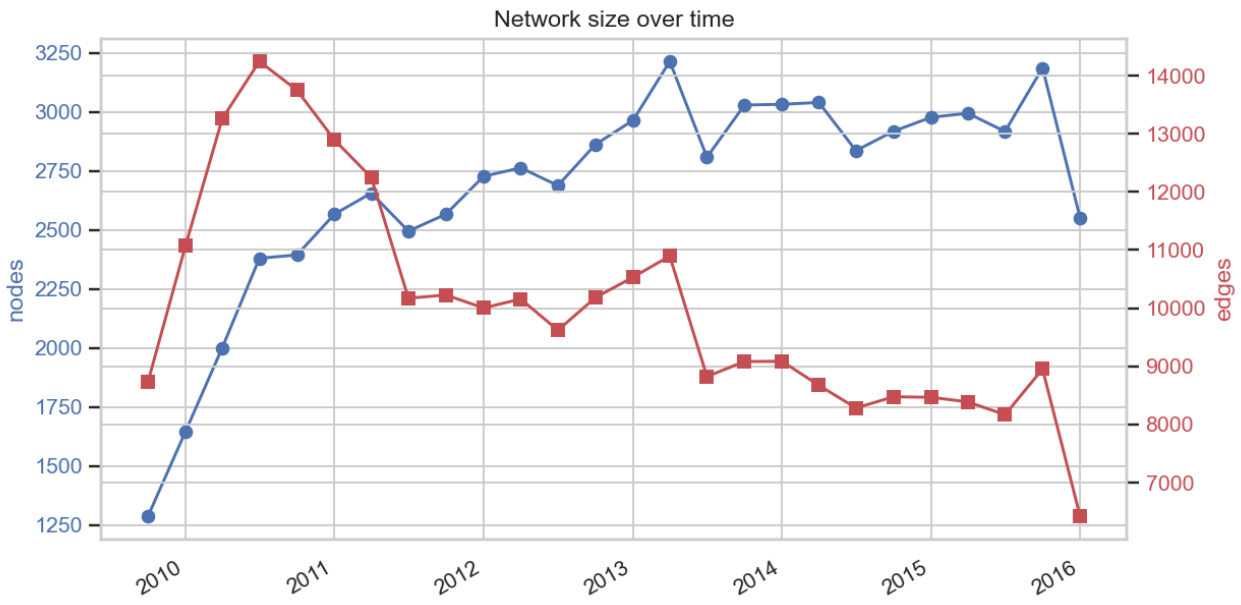
3. Exploratory Network Analysis (Deliverable 2)

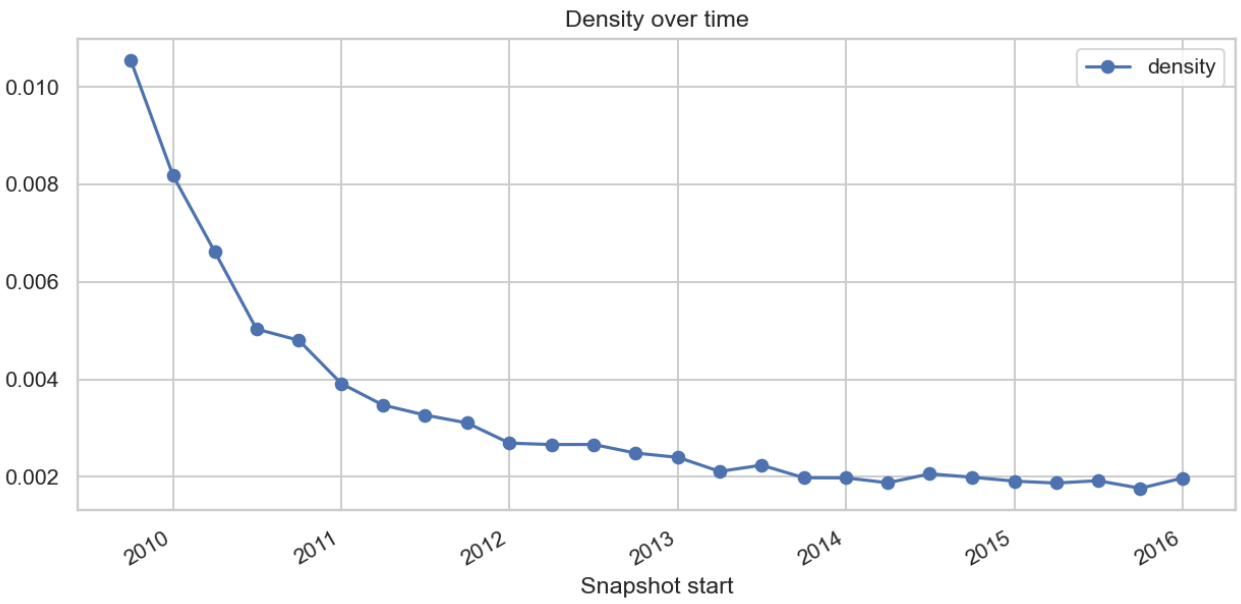
For each snapshot we compute: node count, edge count, density, average degree, average clustering coefficient, transitivity, connected-component count, largest-connected-component size, diameter and average shortest path length on the LCC (sampled for LCC > 500), degree assortativity, and mean/max betweenness centrality (k=200 sampling for n>200).

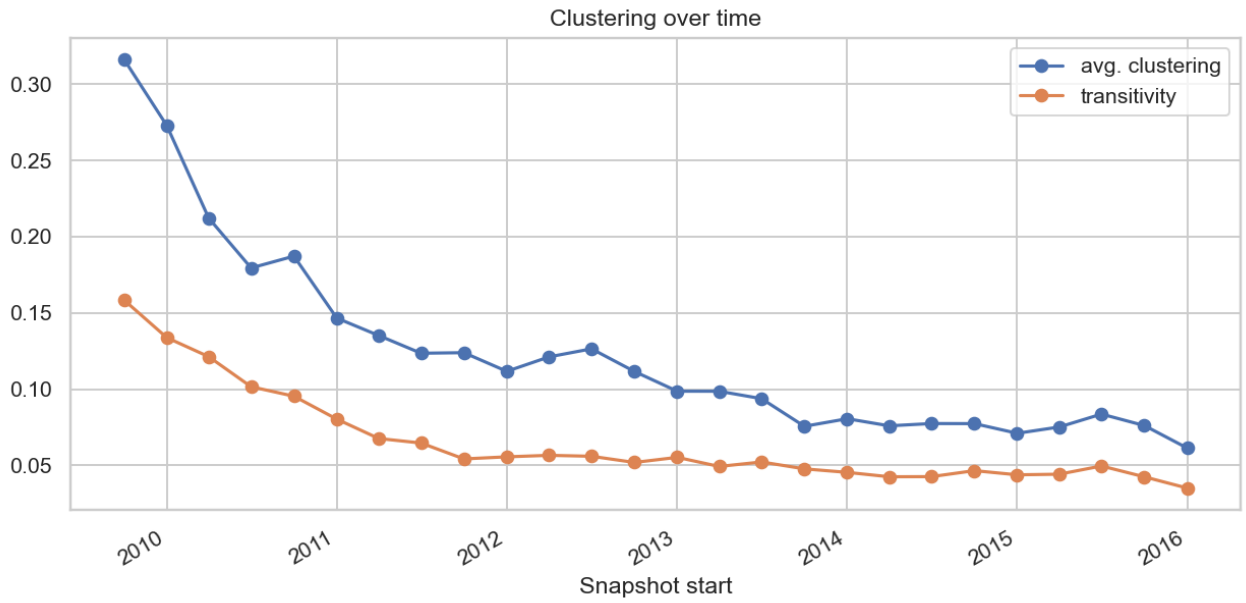
3.1 Headline numbers

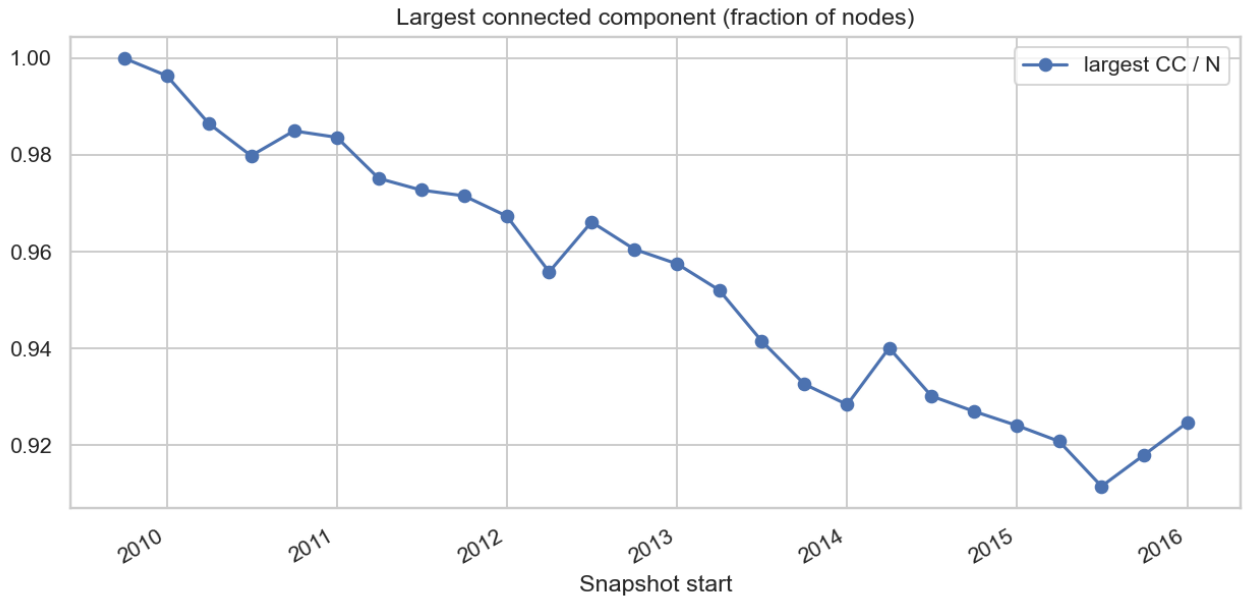
Metric	2009-Q4	2016-Q1	Change
Nodes	1,287	2,550	+98%
Edges	8,731	6,420	–26%
Density	0.0106	0.0020	**–81%**
Avg. degree	13.6	5.0	–63%
Avg. clustering	0.316	0.061	**–81%**

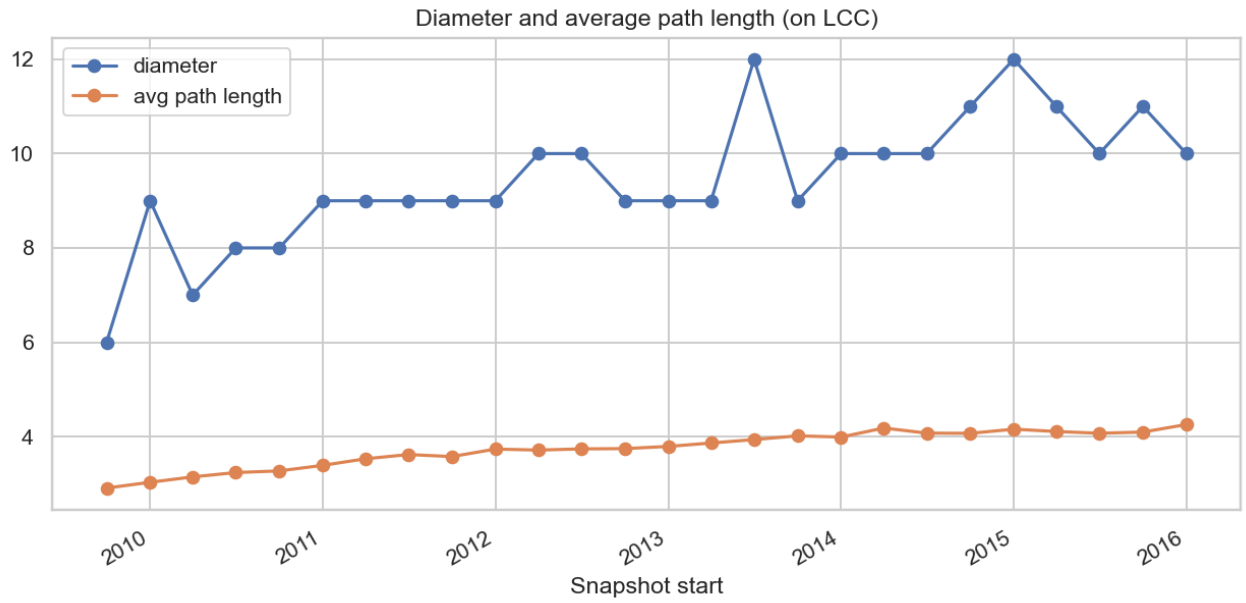
LCC fraction	1.000	0.925	fragmenting
Avg. path length (LCC)	2.91	4.26	lengthening
Diameter (LCC)	6	10	growing
Degree assortativity	−0.18	−0.08	always disassortative

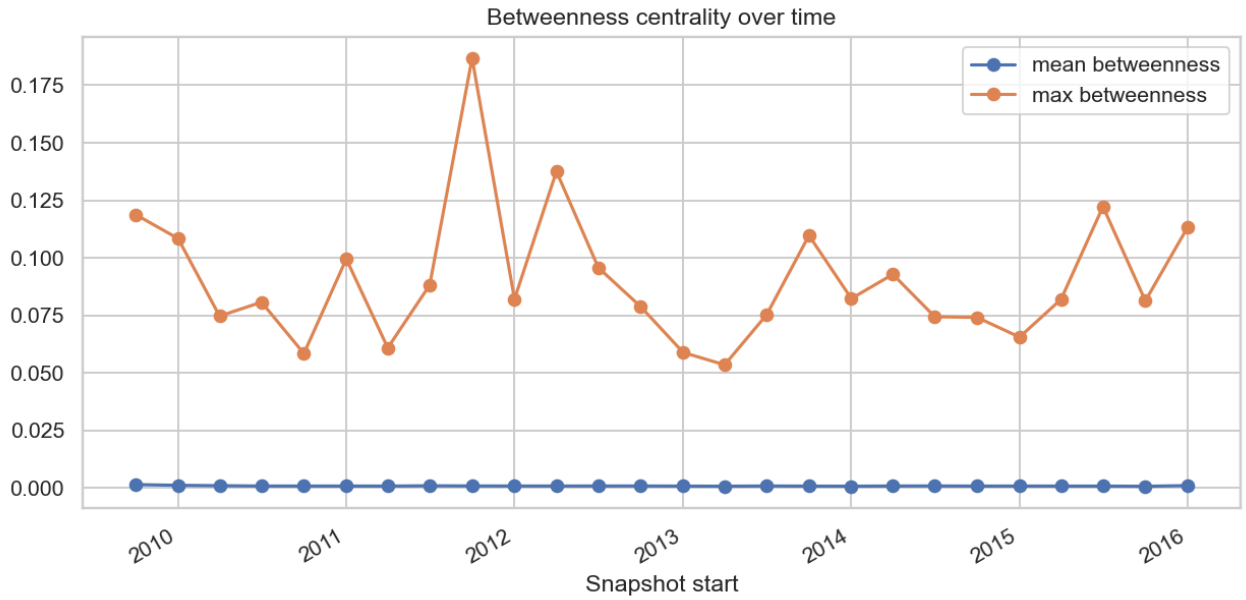


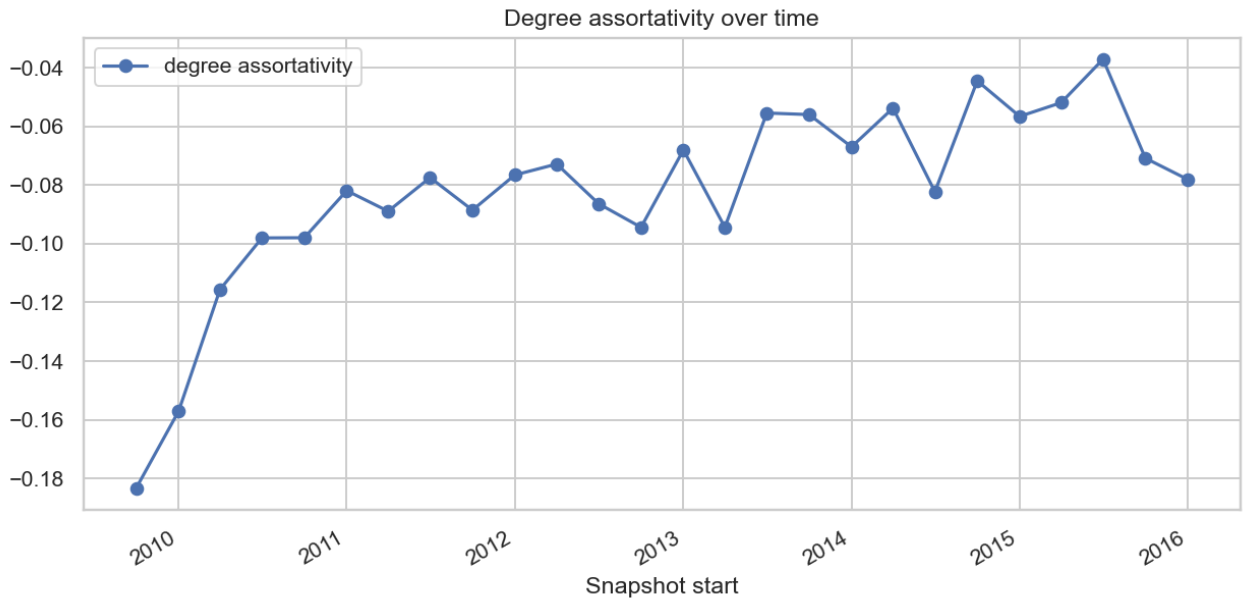


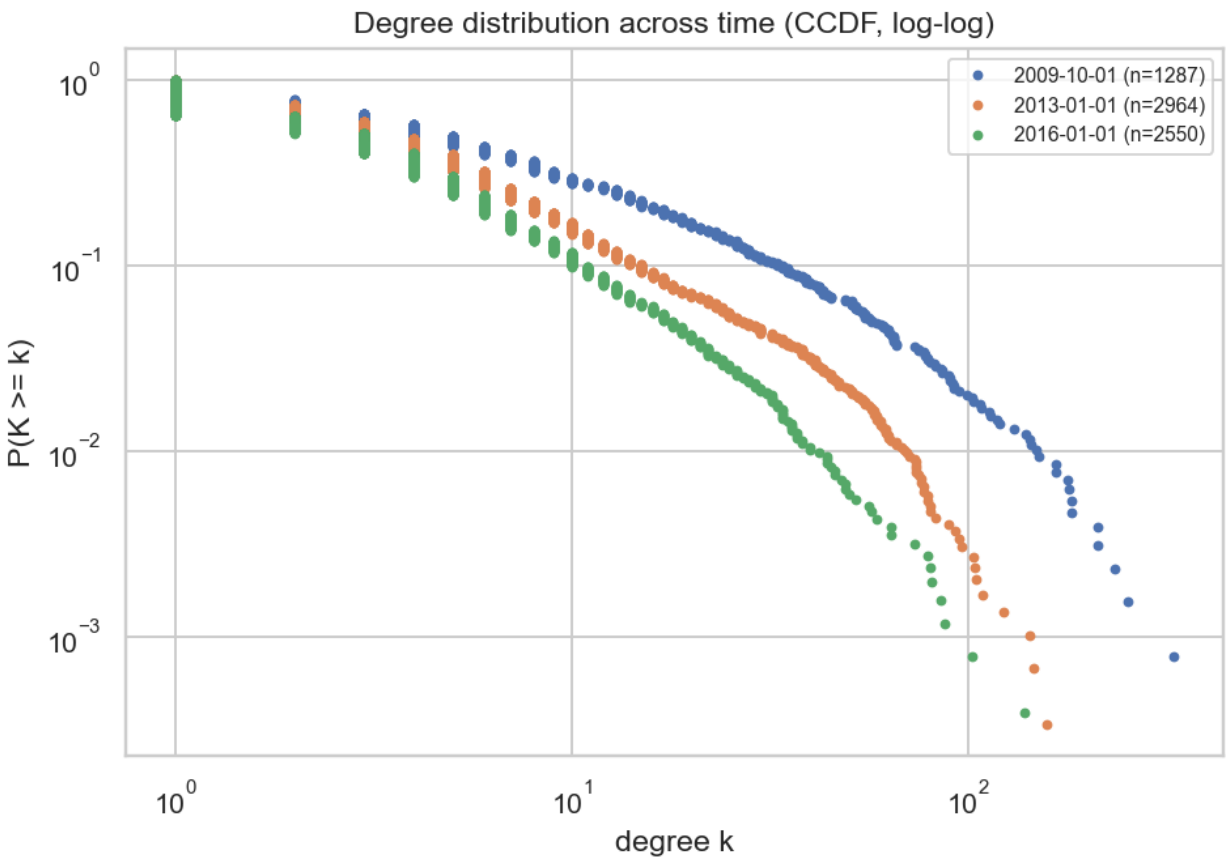

Clustering over time
avg. clustering
transitivity
0.30
0.25
0.20
0.15
0.10
0.05
2010
2011
2012
2013
2014
2015
2016
Snapshot start


Largest connected component (fraction of nodes)
largest CC / N
1.00
0.98
0.96
0.94
0.92
2010
2011
2012
2013
2014
2015
2016
Snapshot start


Diameter and average path length (on LCC)
diameter
avg path length
12
10
8
6
4
2010
2011
2012
2013
2014
2015
2016
Snapshot start


Betweenness centrality over time
mean betweenness
max betweenness
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
2010
2011
2012
2013
2014
2015
2016
Snapshot start





3.2 Interpretation

1. **Sparsification despite growth.** The network roughly doubled in active users but shed edges and lost more than 80% of its density and clustering. This is the classic sparsification pattern for Q&A platforms: early users were a tight community of mathematicians who answered each other; mature-era traffic is dominated by newcomers asking one-off questions and receiving answers from a stable roster of long-lived experts.

2. **Persistent disassortativity.** Degree assortativity is negative in every snapshot — hubs are systematically connected to low-degree nodes, consistent with expert-to-newcomer answer flow.
3. **Growing diameters.** Average path length rising from 2.9 to 4.3 on the LCC indicates that new users are attached peripherally rather than integrating into the expert core.

Implementation: `src/metrics.py`, `notebooks/02_eda.py`.

---

4. Dynamic Community Detection and Event Tracking (Deliverable 3)

4.1 Per-snapshot communities

We apply **Louvain modularity optimization** (python-louvain) to each snapshot independently, using resolution $\gamma=1$ and a fixed random seed for reproducibility. We discard communities with fewer than 5 members.

Modularity stays in the range **0.65–0.70** across all 26 snapshots, indicating strong community structure throughout the dataset's life. The number of communities fluctuates between roughly **25 and 35** per quarter.

4.2 Tracking events across snapshots

We follow the standard Palla-et-al. / Greene-et-al. matching framework: for each pair of consecutive snapshots (S_t, S_{t+1}), we score every pair of communities ($C_a \in S_t$, $C_b \in S_{t+1}$) by **containment overlap** $|C_a \cap C_b| / \min(|C_a|, |C_b|)$. Pairs above a threshold of 0.1 are considered matched. Containment is more appropriate here than Jaccard because each quarterly snapshot contains only users who were active that quarter — a community that "continues" may share only a small fraction of its members across a quarterly boundary since most individual users are not active in every quarter.

From the matching we derive six event types plus a stable "continue" class:

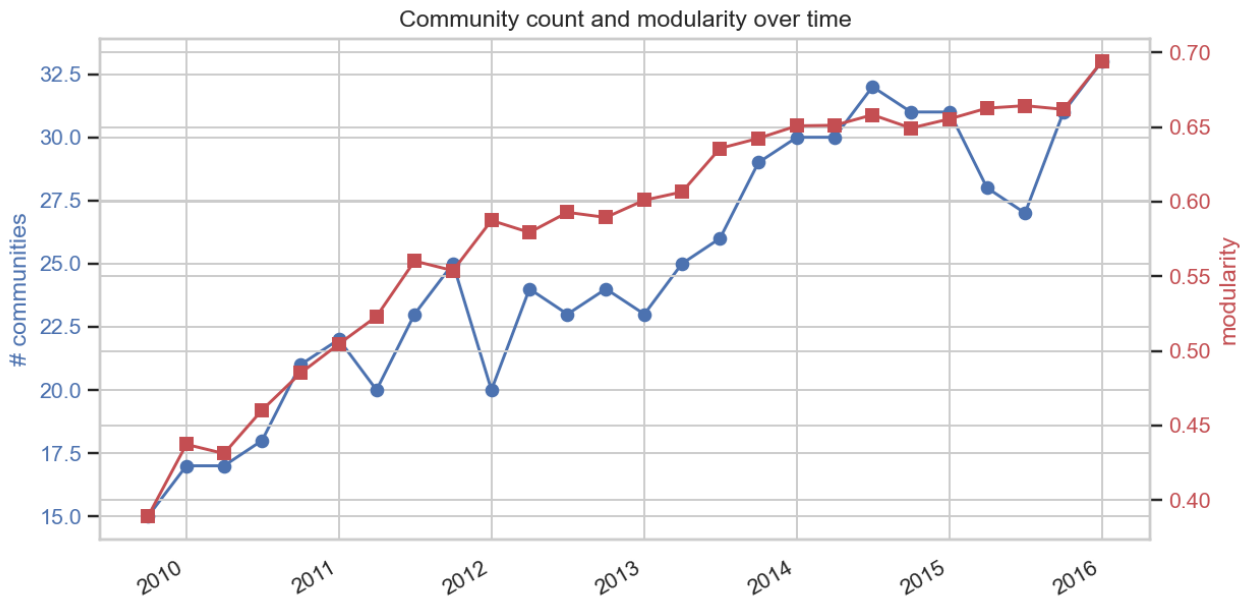
- **birth** — a new community in S_{t+1} matches nothing in S_t
- **death** — a community in S_t matches nothing in S_{t+1}
- **growth / shrinkage** — 1-to-1 match with size ratio above 1.15 or below 0.85
- **continue** — 1-to-1 match with moderate size change
- **merge** — several S_t communities map to one S_{t+1} community
- **split** — one S_t community maps to several S_{t+1} communities

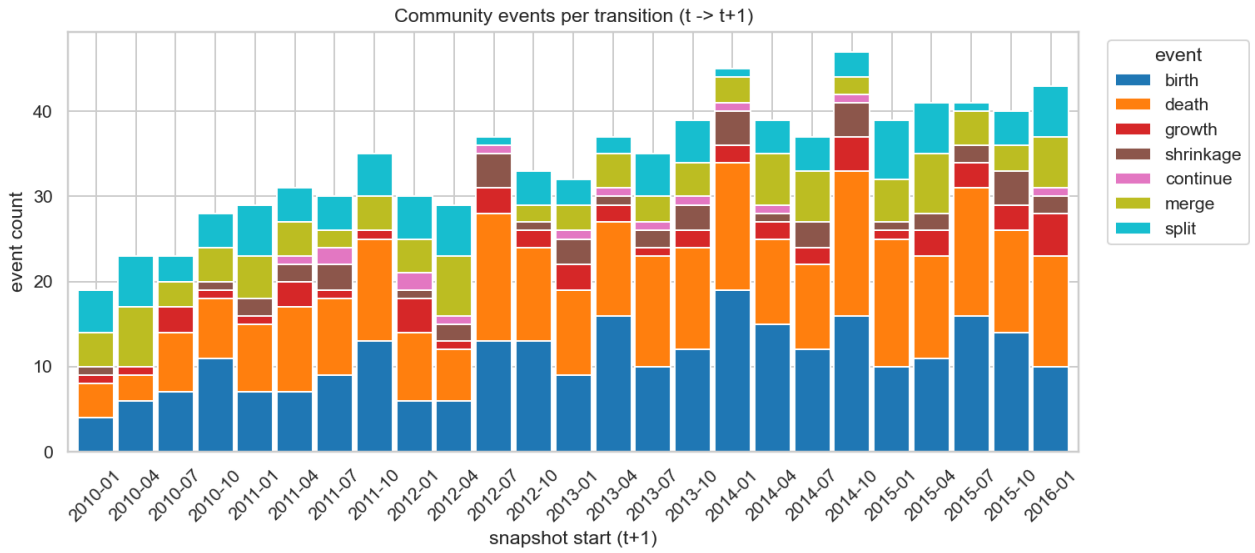
4.3 Findings

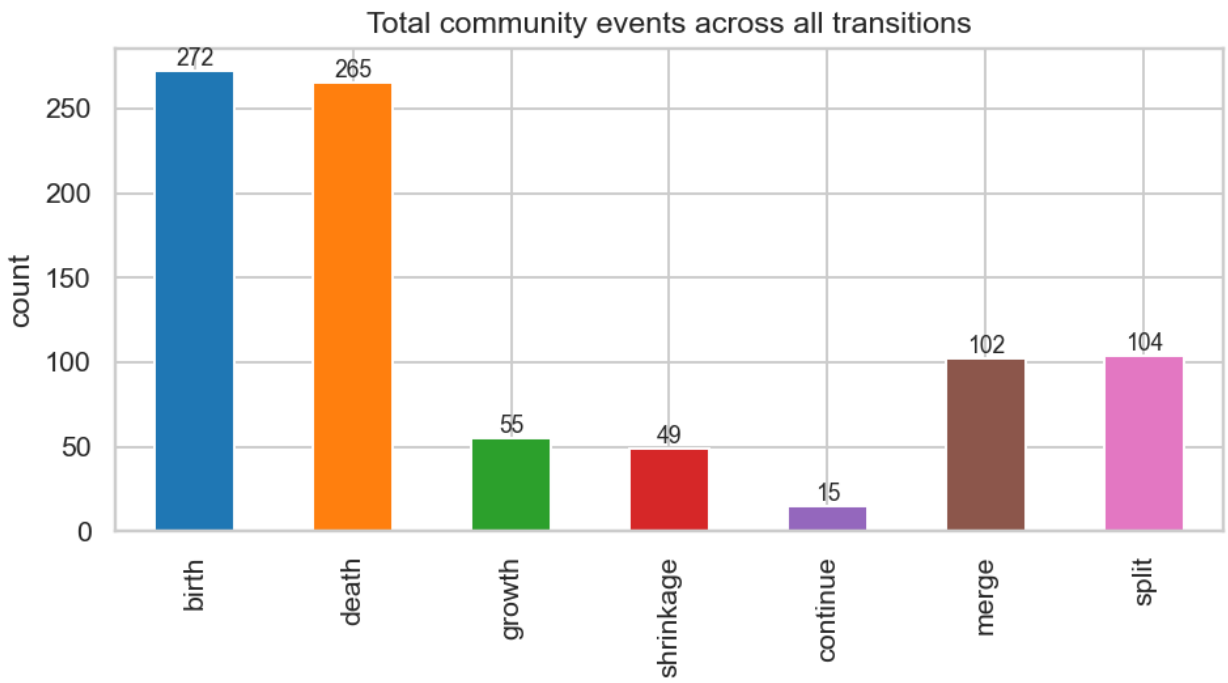
Event	Count	Share
birth	272	31.6%
death	265	30.8%
split	104	12.1%
merge	102	11.8%
growth	55	6.4%
shrinkage	49	5.7%
continue	15	1.7%

Out of **862 events** across 25 transitions (~34 per transition):

- The dominance of **birth (31.6%)** and **death (30.8%)** reflects the quarterly turnover of the active user base on a Q&A platform — many communities are ephemeral clusters around a specific question or event.
- **Merges (11.8%)** and **splits (12.1%)** occur at roughly equal rates, suggesting a steady mixing process where sub-topics coalesce and fracture over time.
- Very few **continue** events (1.7%): a community that persists essentially unchanged from one quarter to the next is rare at this granularity.







Implementation: `src/community.py`, `notebooks/03_communities.py`.

---

5. Link Prediction (Deliverable 4)

5.1 Task setup

We pose the task as **future link prediction** on the cumulative graph.

- Sort all 506,550 edges by timestamp.
- **Training graph**: edges up to the 85th-percentile timestamp (172,350 edges on 20,805 nodes).

- **Test positives**: edges appearing *after* the split between two users both present in the training graph and *not yet connected* there. This yields 13,124 truly new future edges.
- **Test negatives**: 13,124 uniformly sampled non-edges between known nodes that are not future positives.

5.2 Baselines

Four classical similarity heuristics computed on the training graph:

- **Common Neighbors** $|N(u) \cap N(v)|$
- **Jaccard** $|N(u) \cap N(v)| / |N(u) \cup N(v)|$
- **Adamic–Adar** $\Sigma_{w \in N(u) \cap N(v)} 1/\log \deg(w)$
- **Preferential Attachment** $\deg(u) \cdot \deg(v)$

5.3 GraphSAGE

We train a 2-layer **GraphSAGE** in PyTorch Geometric:

- **Input features (64+3 = 67 dim):** a learnable per-node 64-dim embedding table (random init, σ=0.01) concatenated with 3 z-scored structural features (log-degree, clustering coefficient, PageRank).
- **Architecture:** SAGEConv(67 $\rightarrow$ 128) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3) $\rightarrow$ SAGEConv(128 $\rightarrow$ 64).
- **Decoder:** dot product between the two endpoint embeddings.
- **Loss:** BCE over positive training edges vs an equal number of randomly-sampled non-edges, re-sampled each epoch.
- **Optimizer:** Adam, lr 5e-3, weight decay 1e-5, 150 epochs.
- **Hardware:** RTX 5060 (8GB VRAM), CUDA 12.8, PyTorch 2.11.

5.4 Results

Method	AUC	AP
Jaccard	0.857	0.807
Common Neighbors	0.884	0.882
Adamic-Adar	0.886	0.889
GraphSAGE (our model)	0.871	**0.913**
Preferential Attachment	**0.956**	0.957

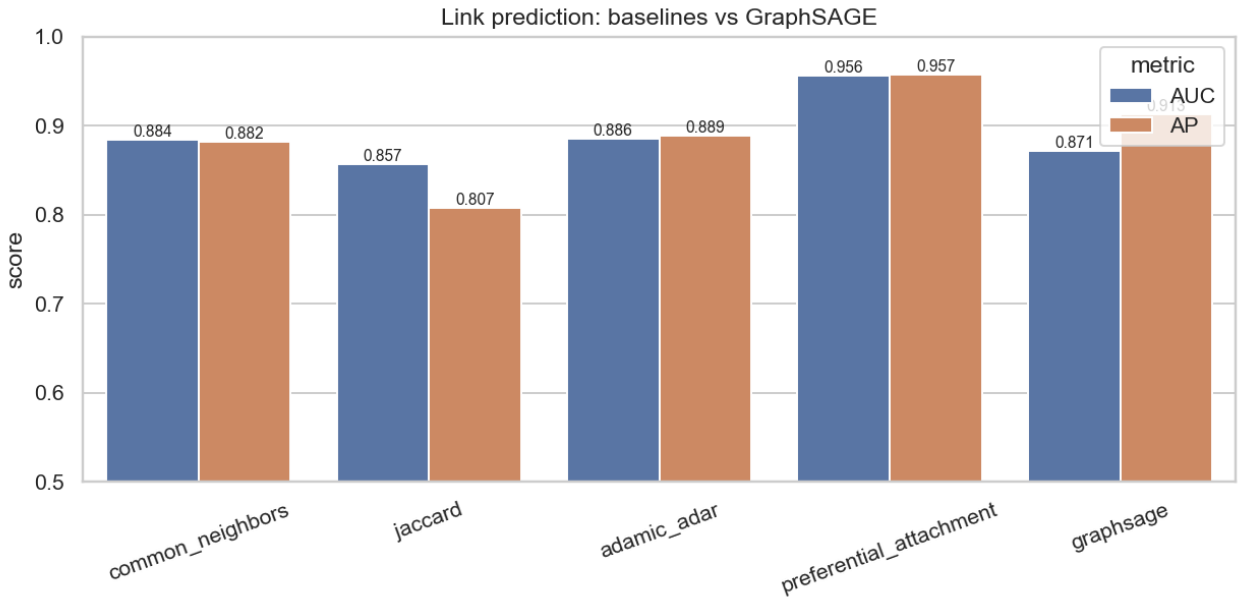
5.5 Discussion

Preferential attachment dominates on both AUC and AP. This is a meaningful (and somewhat humbling) finding for a network science project: the generative mechanism that the Barabási–Albert model has proposed for scale-free networks since 1999 — rich get richer — is so strong in this Q&A graph that a single feature (product of degrees) outperforms a trained graph neural network with ~2M parameters.

GraphSAGE, however, achieves the **second-best AP** (0.913), beating Common Neighbors, Jaccard, and Adamic-Adar. Structural embeddings therefore help with *ranking precision* even when they can't match the raw discriminative power of preferential attachment for Q&A networks.

Why does PA dominate here? The MathOverflow interaction graph is extremely disassortative (hub-to-newcomer) and driven by a few long-lived experts who answer a large fraction of questions. A newcomer's first interaction is overwhelmingly likely to be with one of those hubs. Any predictor that can spot "both endpoints are high-degree" captures most of the signal.

What would help GraphSAGE? Likely candidates: (a) edge weights (interaction counts are available but unused), (b) temporal edges as a feature via a TGN backbone, (c) a concatenated-MLP decoder instead of dot product, and (d) harder negative sampling focused on near-neighbors rather than uniform random pairs.



Implementation: `src/link_prediction.py`, `notebooks/04_link_prediction.py`.

---

6. Interactive Dashboard (Deliverable 5)

A Plotly Dash application at `dashboard/app.py` exposes four interactive tabs:

1. **Network overview** — all global time-series metrics.

2. **Snapshot explorer** — a slider lets the user pick any of the 26 snapshots and renders its largest community as a node-link diagram (spring layout, node size ∝ log-degree, colored by degree), plus a ranked community-size distribution.
3. **Community evolution** — modularity curve, community count, and the full stacked event timeline.
4. **Link prediction** — bar chart of AUC/AP per method.

Run with `python dashboard/app.py` and visit http://127.0.0.1:8050.

---

7. Limitations and Future Work

- **Snapshot granularity.** Quarterly windows lose intra-quarter dynamics; sliding windows (e.g., 12-month windows stepped every month) would improve temporal continuity of communities.
- **Independent-snapshot Louvain.** Running Louvain separately per snapshot produces run-to-run instability in community IDs. Methods like DynComm / dynamic Louvain with temporal smoothing, or temporal stochastic block models, would produce more coherent tracks.
- **GNN scope.** We only tried GraphSAGE with a dot-product decoder on a cumulative training graph. A temporal graph network (TGN) with explicit memory per node could model the temporal dynamics directly rather than collapsing history into a single graph.
- **Interaction types.** MathOverflow mixes three interaction types (a2q, c2q, c2a). Separating them into a multiplex graph is a natural extension.

---

8. Reproducibility

All results in this report can be reproduced with:

```
pip install -r requirements.txt pip install torch --index-url
https://download.pytorch.org/whl/cu128 pip install torch-geometric python
notebooks/01_build_snapshots.py python notebooks/02_eda.py python
notebooks/03_communities.py python notebooks/04_link_prediction.py python
dashboard/app.py
```

Random seeds are fixed (seed=0 for Louvain, PyTorch, and NumPy sampling).

References

(Add the papers actually cited in your final write-up. Candidates:)

- Leskovec, J., Backstrom, L., Kumar, R., & Tomkins, A. (2008). Microscopic evolution of social networks. *KDD*.
- Palla, G., Barabási, A. L., & Vicsek, T. (2007). Quantifying social group evolution. *Nature*.
- Greene, D., Doyle, D., & Cunningham, P. (2010). Tracking the evolution of communities in dynamic social networks. *ASONAM*.
- Blondel, V. D. et al. (2008). Fast unfolding of communities in large networks. *J. Stat. Mech.*
- Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs (GraphSAGE). *NeurIPS*.
- Rossi, E. et al. (2020). Temporal graph networks for deep learning on dynamic graphs. *ICML Workshop*.
- Adamic, L., & Adar, E. (2003). Friends and neighbors on the web. *Social Networks*.
- Liben-Nowell, D., & Kleinberg, J. (2003). The link prediction problem for social networks. *CIKM*.